Neural Networks 205 (2027) 109580

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Full Length Article

DATCNN: A novel CNN network with all the advantages of KAN while offering greater flexibility

Ruikun Luo [a,1], Nan Su [a,1], Yixiang Dai [a], Guosheng Li [a], Guijin Wang [a,b,*]

[a] Department of Electronic Engineering, Tsinghua University, Beijing, 100084, China
[b] Shanghai AI Laboratory, Shanghai, 200232, China

ARTICLE INFO

Keywords:
XAI
CNN

ABSTRACT

In recent years, the interpretability of artificial intelligence models has been increasingly valued. Kolmogorov–Arnold Network (KAN) is a novel neural network architecture that supports symbolic regression and offers good interpretability. Derivative works of KAN not only provide interpretability but also demonstrate good performance in common tasks such as image segmentation and time series prediction. However, KANs suffer from slow convergence and difficult training processes. To address these limitations, we propose the Dense Affine Transformation Convolutional Neural Network (DATCNN). This novel CNN-based architecture preserves the interpretability and functional representation capacity of KANs while offering enhanced flexibility and compatibility with established CNN theory and training heuristics. Experimental results across multiple tasks, including function fitting, image classification, and natural language processing, demonstrate that DATCNN achieves faster training speeds and superior performance, highlighting its potential as an efficient alternative to KANs in theoretical and practical settings.

1. Introduction

The Kolmogorov-Arnold representation theorem demonstrates that any bounded multivariate continuous function can be expressed as a finite composition of univariate functions and addition, thereby providing theoretical guidance for the representational capacity of neural networks (Hornik et al., 1989). Multilayer perceptrons (MLPs) (Cybenko, 1989; Haykin, 1998; Hornik et al., 1989) are among the fundamental building blocks of contemporary deep neural networks. MLPs have given rise to numerous advanced network architectures such as convolutional neural networks (CNNs) (He et al., 2016; Krizhevsky et al., 2012) and transformers (Vaswani et al., 2017), all of which have found extensive applications in fields like computer vision and natural language processing. However, these models are typically characterized by their complex and nested structures, which result in poor interpretability. The potential faults within these models are challenging to detect in advance. In some scenarios, they may lead to catastrophic consequences, posing legal and ethical risks (Zhang et al., 2021). In recent years, researchers have proposed numerous methods related to model interpretability, which can be categorized into four types based on the format of the generated explanations (Zhang et al., 2021): examples, attributions, hidden semantics, and rules. Example-based explanations provide additional supporting or refuting examples for any new input, such as the most similar examples or representative sample sets within each category (Bien & Tibshirani, 2011; Caruana et al., 1999). Attribution-based explanations indicate the influence of each input feature on the output, for instance, saliency maps (Selvaraju et al., 2017; Simonyan et al., 2013; Springenberg et al., 2014; Zeiler & Fergus, 2014). Explanations based on hidden semantics focus on the meanings of hidden neurons or layers, attempting to link abstract concepts with the activation of specific hidden neurons (Erhan et al., 2009; Simonyan et al., 2013). Whether it is examples, attributions, or hidden semantics, the explanations for models are indirect. In contrast, rule-based explanations offer the most direct explanations by providing a set of rules or decision trees equivalent to the model (Setiono & Liu, 1995; Wang et al., 2018b). However, the representational capacity of rule sets and decision trees is limited, making it challenging to describe mathematical functions. To overcome this drawback, Symbolic regression methods have gradually replaced rule-based methods and become the mainstream of explanation solutions, as they possess stronger representational capabilities and can provide mathematical expressions (Makke & Chawla, 2024; Udrescu & Tegmark, 2020). Nevertheless, most existing symbolic regression

* Corresponding author.
E-mail address: wangguijin@tsinghua.edu.cn (G. Wang).
[1] Equal Contribution.

https://doi.org/10.1016/j.neunet.2026.109580
Received 10 July 2025; Received in revised form 14 August 2026; Accepted 30 August 2026
Available online 2 September 2026

methods are search-based and directly target end-to-end data, resulting in a rapidly growing search space as the model size increases, making it difficult to handle complex models.

The Kolmogorov-Arnold Network (KAN) (Liu et al., 2025) is a novel network architecture that supports symbolic regression and offers good interpretability. KAN is composed of many learnable activation functions. During symbolic regression, KAN can search for each activation function separately, avoiding the vast search space problem. Despite the promising potential of KAN in many scenarios, slow convergence and difficult training have always been significant drawbacks. Due to the use of learnable activation functions, KAN exhibits an essential structural difference from MLPs and their derivatives, such as CNNs and transformers. As a result, KAN lacks flexibility, meaning that existing methods for accelerating model training are challenging to transfer directly to KAN, and the effectiveness of any transferred methods lacks both theoretical and empirical guarantees.

In recent years, researchers have proposed new learnable activation functions to improve the training speed of KAN (Aghaei, 2024, 2025; Sidharth et al., 2024). fKAN (Aghaei, 2025) is the most advanced variant of KAN, replacing the ordinary activation functions in MLP with trainable Jacobi basis functions. However, the performance of learnable activation functions is directly related to the form of the ground truth. Each learnable activation function has its specific application scenarios and does not have universal advantages. Moreover, these methods still have the drawback of insufficient flexibility. PowerMLP (Qiu et al., 2025) is a distinct variant of KAN that employs fixed higher-order functions, ReLU^k, as activation layers within an MLP. While this modification reduces computational overhead, it incurs a performance trade-off. In addition to modifying the learnable activation functions of KAN, researchers have also widely applied KAN to various common tasks, such as image segmentation (Cen et al., 2025; Li et al., 2025), image classification (Bodner et al., 2024), time series prediction (Genet & Inzirillo, 2024), and graph problems (Bresson et al., 2024; Taghizadeh et al., 2025). These approaches can be summarized as either inserting KAN as an independent module into the model or replacing the MLP module in the original model with KAN, thereby helping the model better distinguish highly nonlinear local latent concepts (Li et al., 2025). For example, Convolutional KAN (Bodner et al., 2024) replaces the generalized linear module in convolutional layers with KAN, thereby improving image classification performance. However, this also brings KAN's drawbacks into the models, leading to slower training speeds and reduced flexibility.

In this paper, we propose a novel convolutional neural network structure, called the Dense Affine Transformation Convolutional Neural Network (DATCNN), to address the lack of uniformity between KAN and mainstream CNN architectures. Each DATCNN layer employs a dual-path architecture comprising a backbone and a residual connection. Its backbone is formed by stacking multiple convolutional layers with batch normalization and activation, followed by a fully connected layer without activation. By applying affine transformations to activation functions within the convolutional layers, DATCNN achieves the effect of learnable activation functions in KAN, thereby facilitating symbolic regression and enhancing interpretability. Compared to KAN, DATCNN offers superior flexibility by effectively leveraging established CNN theories and empirical design principles. Overall, DATCNN preserves the interpretability benefits of KAN while providing greater flexibility.

We theoretically validate the DATCNN architecture through mathematical analysis and simulation experiments. Its performance is comprehensively evaluated across diverse scenarios, including function fitting, image classification, and natural language processing. Experimental results demonstrate that DATCNN outperforms KAN and its variants in both accuracy and training efficiency, highlighting its flexibility and broad application potential. Our key contributions are summarized as follows.

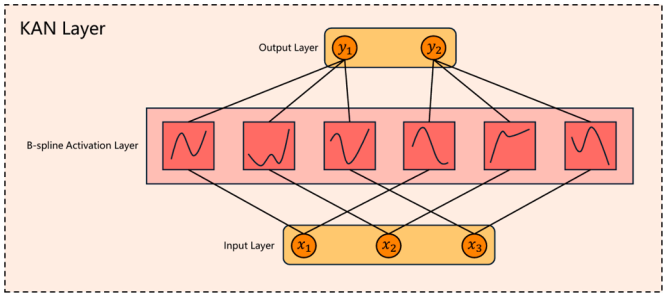


Fig. 1. The structural diagram of a KAN layer. A KAN layer with input dimension d_I and output dimension d_O has a total of $d_I d_O$ learnable activation functions. Each output is formed by summing the inputs after each passes through a learnable activation function.

- We propose DATCNN, a novel convolutional neural network architecture that inherits the core strengths of KAN while offering significantly enhanced flexibility.
- We formulate a mathematical theory for DATCNN, theoretically guaranteeing its capabilities in symbolic regression and interpretability.
- Extensive experiments demonstrate that DATCNN outperforms KAN and other state-of-the-art variants, achieving superior accuracy with reduced training latency.

2. Kolmogorov-Arnold networks

The inspiration for KAN originates from the Kolmogorov-Arnold representation theorem, which states that any bounded multivariate continuous function can be expressed as a finite composition of univariate functions and addition:

$$f(x_1, x_2, \ldots, x_n) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^{n} \phi_{q,p}(x_p) \right). \tag{1}$$

KAN observes that the right-hand side of Eq. (1) can be viewed as stacking two layers, thereby generalizing the above representation theorem to wider and deeper cases. Fig. 1 illustrates the structure of a single KAN Layer. Specifically, a single KAN Layer Φ_l can be regarded as a matrix composed of several univariate functions:

$$\Phi_l = \begin{bmatrix} \phi_{l,1,1} & \phi_{l,1,2} & \cdots & \phi_{l,1,n_l} \\ \phi_{l,2,1} & \phi_{l,2,2} & \cdots & \phi_{l,2,n_l} \\ \vdots & \vdots & \ddots & \vdots \\ \phi_{l,n_{l+1},1} & \phi_{l,n_{l+1},2} & \cdots & \phi_{l,n_{l+1},n_l} \end{bmatrix}, \tag{2}$$

And the input-output relationship of the KAN Layer can be expressed in the form of matrix multiplication:

$$\begin{aligned} \Phi_l(x) &= \begin{bmatrix} \phi_{l,1,1} & \phi_{l,1,2} & \cdots & \phi_{l,1,n_l} \\ \phi_{l,2,1} & \phi_{l,2,2} & \cdots & \phi_{l,2,n_l} \\ \vdots & \vdots & \ddots & \vdots \\ \phi_{l,n_{l+1},1} & \phi_{l,n_{l+1},2} & \cdots & \phi_{l,n_{l+1},n_l} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{n_l} \end{bmatrix} \\ &= \begin{bmatrix} \sum_{p=1}^{n_l} \phi_{l,1,p}(x_p) \\ \sum_{p=1}^{n_l} \phi_{l,2,p}(x_p) \\ \vdots \\ \sum_{p=1}^{n_l} \phi_{l,n_{l+1},p}(x_p) \end{bmatrix}. \end{aligned} \tag{3}$$

A typical KAN is a stacking of several KAN Layers. For example, an L-layer KAN can be expressed as:

$$\mathrm{KAN}(x) = (\Phi_L \circ \Phi_{L-1} \circ \cdots \circ \Phi_1)(x). \tag{4}$$

The ability of KAN to learn activation functions directly is implemented by the B-spline:

$$\phi(x) = wb \cdot \mathrm{SiLU}(x) + ws \cdot \sum_{v=1}^{n} c_v B_{\xi,p,v}(x). \tag{5}$$

where wb, ws, c_ν are weights, and $B_{\xi,p,\nu}$ is a set of functions known as B-splines. Their definition is recursive, as shown in Definition 1 (Fakhoury et al., 2022).

Definition 1. For any natural number p and an increasing finite real sequence ξ (length $n+p+1$), a set of real functions $B_{\xi,p,\nu}$ can be recursively defined as:

$$B_{\xi,p,\nu}(x) = \frac{x-\xi_\nu}{\xi_{\nu+p}-\xi_\nu} B_{\xi,p-1,\nu}(x) + \frac{\xi_{\nu+p+1}-x}{\xi_{\nu+p+1}-\xi_{\nu+1}} B_{\xi,p-1,\nu+1}(x), \tag{6}$$

$$B_{\xi,0,\nu}(x) = \begin{cases} 1 & x \in [\xi_\nu, \xi_{\nu+1}), \\ 0 & \text{otherwise.} \end{cases} \tag{7}$$

The sequence ξ in Definition 1 is called the knot vector, an essential parameter of KAN. However, in KAN, the optimization of ξ is based on rules rather than gradient descent, which is one of the main reasons for the slow convergence of KAN. Another important reason is the high computational complexity of B-splines themselves. It can be seen that B-splines provide KAN with the ability to learn activation functions directly; however, they also cause KAN to exhibit slow convergence and difficult training, leaving much room for improvement.

To facilitate subsequent mathematical arguments, we formally define the KAN Layer as shown in Definition 2.

Definition 2. For any natural number p, finite real sequence ξ (length $n+p+1$), and real tensors wb, ws (shape (d_O, d_I)), c (shape (d_O, d_I, n)), the KAN Layer is a mapping from $\mathbb{R}^{d_I}$ to $\mathbb{R}^{d_O}$:

$$[\mathrm{KANLayer}(x;\xi,p,wb,ws,c)]_i = \sum_{j=0}^{d_I-1} \phi_{ij}(x_j;\xi,p,wb_{ij},ws_{ij},c_{ij}), \tag{8}$$

where:

$$\phi_{ij}(x_j;\xi,p,wb_{ij},ws_{ij},c_{ij}) = wb_{ij}\cdot \mathrm{SiLU}(x_j) + ws_{ij}\cdot \mathrm{spline}_{ij}(x_j;\xi,p,c_{ij}), \tag{9}$$

$$\mathrm{spline}_{ij}(x_j;\xi,p,c_{ij}) = \sum_{\nu=0}^{n-1} c_{ij\nu} B_{\xi,p,\nu}(x_j). \tag{10}$$

3. Method

This section presents DATCNN and elucidates its theoretical underpinnings. As illustrated below, DATCNN is derived from a novel convolutional neural network architecture equivalent to KAN, incorporating specific enhancements. Section 3.1 provides a detailed structure of DATCNN. Section 3.2 analyzes the translational symmetry of B-splines in KAN. Building on Section 3.2, Section 3.3 defines a new function called the normalized B-function, which makes the computation of B-splines more convenient and flexible. Further, Section 3.3 transforms the normalized B-function into a non-recursive form and provides a method for calculating the coefficients. Section 3.4 offers a novel perspective on the findings in Section 3.3 and designs a convolutional network structure equivalent to KAN. Section 3.5 refines this architecture while preserving its symbolic regression capabilities, culminating in the proposal of DATCNN in Section 3.1.

3.1. Dense affine transformation convolutional neural network

The proposed DATCNN consists of several stacked DATCNN layers, as shown in Fig. 2. A DATCNN layer employs a dual-path structure, consisting of a backbone and a residual connection. Its backbone is formed by stacking multiple convolutional layers with batch normalization and activation, followed by a fully connected layer without activation. When the input and output dimensions are the same, the residual path uses the identity mapping; when they differ, it uses a fully connected layer. In practice, all batch normalization and the activation layers in the residual path are optional. Definition 3 provides a formal description of the DATCNN layer.

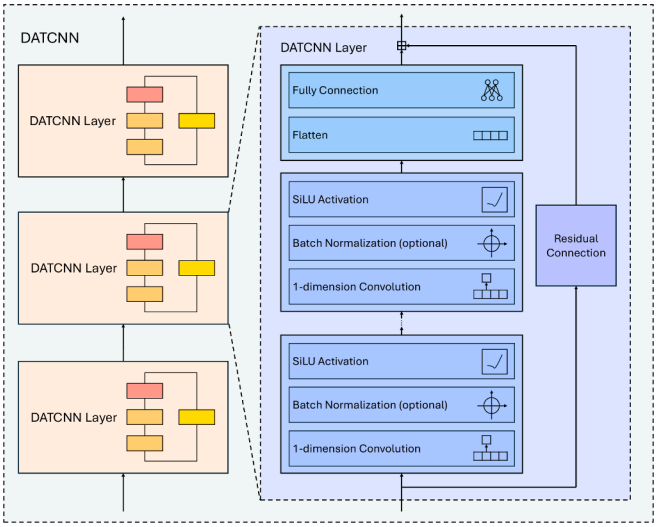


Fig. 2. Structural diagram of DATCNN. A DATCNN layer uses a dual-path structure, consisting of a backbone and a residual connection. Its backbone is formed by stacking multiple convolutional layers with batch normalization and activation, followed by a fully connected layer without activation at the end.

Definition 3. For input dimension d_I and output dimension d_O, the DATCNNLayer is a mapping from $\mathbb{R}^{d_I}$ to $\mathbb{R}^{d_O}$, as shown in Eq. (11), where FC denotes a fully connected layer, Act denotes an activation layer, BatchNorm denotes a batch normalization layer, Conv denotes a convolutional layer, and Residual denotes a residual connection:

$$\mathrm{DATCNNLayer}(x) = \mathrm{FC}\circ\mathrm{Act}\circ\mathrm{BatchNorm}\circ\mathrm{Conv}\circ\cdots\circ\mathrm{Act}\circ\mathrm{BatchNorm}\circ\mathrm{Conv}(x) + \mathrm{Residual}(x). \tag{11}$$

It is worth noting that the convolution in DATCNN is typically dense, that is, the number of convolutional kernels is significantly greater than the dimensions of both the input and the output. The composition of convolution and activation functions can be viewed as an affine transformation of the activation functions. Numerous activation functions transformed by affine transformations can be the basis for learnable activation functions, thereby enabling DATCNN to possess symbolic regression capabilities.

3.2. Properties of B-splines

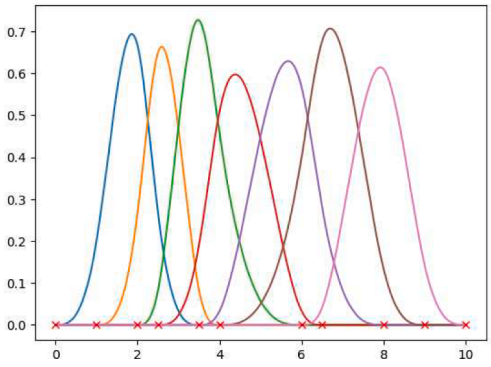
Fig. 3 illustrates B-splines, where Fig. 3a shows a general B-spline, and Fig. 3b shows a special case with uniformly spaced knots. In this special case, the shapes of $B_{\xi,p,\nu}$ for different indices ν are identical, differing only in position. This conclusion can be formally stated and proven by Theorem 1.

Theorem 1. *For a B-spline with parameters p and ξ (length $n+p+1$), if ξ is uniform, i.e.,*

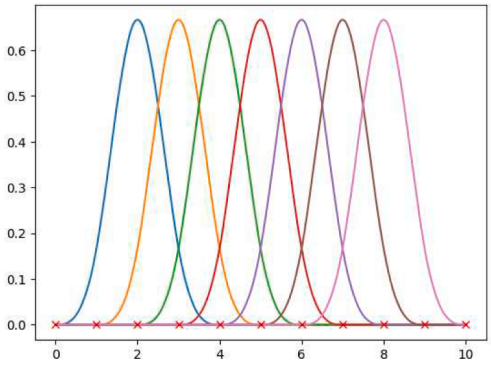
$$\forall \nu \in \mathbb{Z} \cap [0, n-1] \quad \xi_\nu = \xi_0 + \nu\delta, \tag{12}$$

then:

$$\forall \nu \in \mathbb{Z} \cap [0, n-1] \quad B_{\xi,p,\nu}(x) = B_{\xi,p,0}(x - \nu\delta). \tag{13}$$



(a) General B-spline.



(b) Uniform B-spline.

Fig. 3. Illustration of B-splines. Fig. 3a shows a general B-spline, and Fig. 3b shows a special case with uniformly spaced knots, where the shapes of $B_{\xi,p,\nu}$ for different indices ν are identical, differing only in position.

Proof. We use induction on p. When $p = 0$, the statement is true. Assume it holds for some natural number p, and consider the case for $p + 1$:

$$\begin{aligned} &B_{\xi,p+1,\nu}(x) \\ &= \frac{x - \xi_\nu}{\xi_{\nu+p+1} - \xi_\nu} B_{\xi,p,\nu}(x) + \frac{\xi_{\nu+p+2} - x}{\xi_{\nu+p+2} - \xi_{\nu+1}} B_{\xi,p,\nu+1}(x) \\ &= \frac{x - \nu\delta - \xi_0}{(p+1)\delta} B_{\xi,p,0}(x - \nu\delta) \\ &\quad + \frac{\xi_0 + (p+2)\delta - (x - \nu\delta)}{(p+1)\delta} B_{\xi,p,0}(x - \nu\delta - \delta), \end{aligned} \tag{14}$$

Substituting $\nu = 0$ gives:

$$\begin{aligned} B_{\xi,p+1,0}(x) = &\frac{x - \xi_0}{(p+1)\delta} B_{\xi,p,0}(x) \\ &+ \frac{\xi_0 + (p+2)\delta - x}{(p+1)\delta} B_{\xi,p,0}(x - \delta), \end{aligned} \tag{15}$$

It is easy to see that $B_{\xi,p+1,\nu}(x) = B_{\xi,p+1,0}(x - \nu\delta)$. □

Although strict translational symmetry only holds for uniform knots, Fig. 3a shows that even when the knots are not uniform, the components of the B-spline are still similar smooth pulse curves. One component can be approximately transformed into another by horizontal stretching and translation, as well as vertical scaling, which can be implemented using the convolutional and fully connected layers in DATCNN. In the subsequent mathematical derivations, we only discuss the case of uniform knots, but the conclusion we obtain is also approximately valid for non-uniform knots. The correctness of this generalization will be verified in the experimental section.

3.3. Normalized B-function

To compute B-splines more efficiently, we define the normalized B-function as the 0th B-function when $\xi_n = n$, as shown in Definition 4. With the help of Proposition 1, we present a method to represent B-splines using the normalized B-function, as shown in Theorem 2.

Definition 4. The definition of the pth order normalized B-function is:

$$\hat{B}_p(x) = B_{\hat{\xi},p,0}(x) \quad \text{where} \quad \hat{\xi}_\nu = \nu. \tag{16}$$

Proposition 1. *The pth order B-function satisfies the following recursive relationship:*

$$\hat{B}_p(x) = \frac{x}{p} \hat{B}_{p-1}(x) + \frac{p + 1 - x}{p} \hat{B}_{p-1}(x - 1), \tag{17}$$

$$\hat{B}_0(x) = \begin{cases} 1 & \text{if } x \in [0, 1), \\ 0 & \text{otherwise.} \end{cases} \tag{18}$$

Proof. Substituting $\xi_\nu = \nu$ and $\nu = 0$ into Definition 1 yields the result. □

Theorem 2. *If ξ is uniform, then the B-spline can be expressed using the normalized B-function. Let the interval be δ, then:*

$$B_{\xi,p,\nu}(x) = \hat{B}_p\left(\frac{x - \xi_0}{\delta} - \nu\right). \tag{19}$$

Proof. For Definition 1, taking $\xi_\nu = \xi_0 + \nu\delta$ and $\nu = 0$, and replacing x with $\delta x + \xi_0$, we obtain:

$$\begin{aligned} &B_{\xi,p,0}(\delta x + \xi_0) \\ &= \frac{x}{p} B_{\xi,p-1,0}(\delta x + \xi_0) \\ &\quad + \frac{p + 1 - x}{p} B_{\xi,p-1,0}[\delta(x - 1) + \xi_0], \end{aligned} \tag{20}$$

$$B_{\xi,0,0}(\delta x + \xi_0) = \begin{cases} 1 & \text{if } x \in [0, 1), \\ 0 & \text{otherwise.} \end{cases} \tag{21}$$

It can be seen that $B_{\xi,p,0}(\delta x + \xi_0) = \hat{B}_p(x)$, since their recursive formulas are identical. Using Theorem 1 and replacing x with $\frac{x - \xi_0}{\delta} - \nu$ completes the proof. □

Theorem 2 is significant because it shows that any B-spline with arbitrary parameters can be represented using a deterministic normalized B-function. This decouples the recursive structure in the definition of B-splines from the knot points, thereby allowing for simplified computation. Proposition 2 further provides a method for computing the normalized B-function.

Proposition 2. *The normalized B-function $\hat{B}_p$ can be expressed as a piecewise function:*

$$\hat{B}_p(x) = \begin{cases} A_{p,0}(x) & \text{if } x \in [0, 1), \\ A_{p,1}(x) & \text{if } x \in [1, 2), \\ \dots & \dots \\ A_{p,p}(x) & \text{if } x \in [p, p + 1). \end{cases} \tag{22}$$

Here, $A_{p,\mu}$ is a polynomial of degree p:

$$A_{p,\mu}(x) = \frac{x}{p} A_{p-1,\mu}(x) + \frac{p + 1 - x}{p} A_{p-1,\mu-1}(x - 1), \tag{23}$$

$$A_{0,0}(x) = 1. \tag{24}$$

Proof. We use induction on p. When $p = 0$, the statement is true. Assume it holds for some natural number p, and consider the case for $p + 1$. Substituting into Theorem 2 completes the proof. □

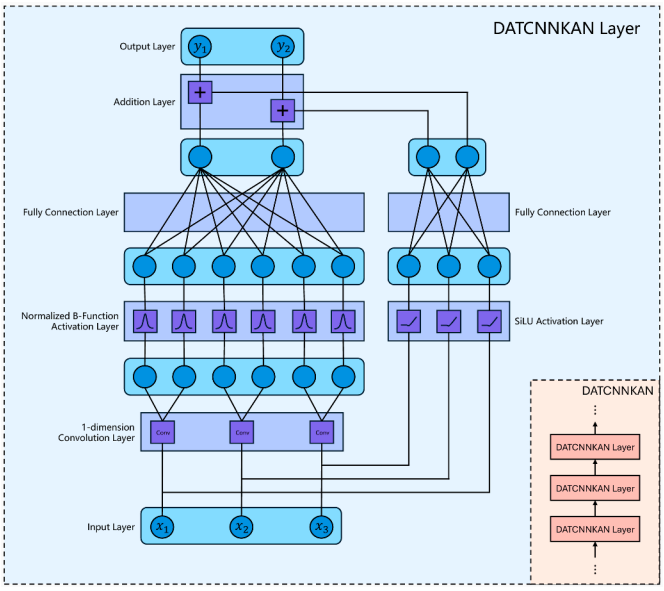


Fig. 4. Structural diagram of DATCNNKAN. DATCNNKAN is composed of multiple stacked DATCNNKAN layers. The backbone of a DATCNNKAN layer consists of three parts: a one-dimensional convolution layer with kernel size 1, a normalized B-function activation layer, and a fully connected layer. In addition to the backbone, the DATCNNKAN layer also includes a residual connection, which is composed of a SiLU activation layer and a fully connected layer.

The coefficients of each term in $A_{p,\mu}$ are independent of the input and can be computed once during function instantiation. For example, $\hat{B}_2$ can be represented as a piecewise quadratic polynomial:

$$\hat{B}_2(x) = \begin{cases} \frac{1}{2}x^2 & x \in [0,1), \\ -x^2 + 3x - \frac{3}{2} & x \in [1,2), \\ \frac{1}{2}x^2 - 3x + \frac{9}{2} & x \in [2,3). \end{cases} \tag{25}$$

Therefore, $\hat{B}_p$ remains fixed during training and inference, thereby reducing computational cost.

3.4. A convolutional network architecture equivalent to KAN

Theorem 2 demonstrates that a normalized B-function can be transformed into a B-spline through scaling and translation. As indicated by the right-hand side of Eq. (19), these operations can be implemented via a one-dimensional convolution with a kernel size of 1. Consequently, a convolutional network architecture can replicate the computation graph of KAN. We denote this proposed architecture as DATCNNKAN. As illustrated in Fig. 4, the model consists of stacked DATCNNKAN layers. The backbone of each layer comprises three components: a one-dimensional convolution, a normalized B-function activation, and a fully connected layer. Additionally, the layer includes a residual connection comprising a SiLU activation and a fully connected layer. The formal definition is provided in Definition 5, and its equivalence to KAN is rigorously established in Theorem 3.

Definition 5. For any real tensors wsc (shape (d_O, nd_I)) and wb (shape (d_O, d_I)), an integer p, and real numbers ξ_0 and δ, the DATCNNKAN layer is a mapping from $\mathbb{R}^{d_I}$ to $\mathbb{R}^{d_O}$:

$$\begin{aligned} &\mathrm{DATCNNKANLayer}(x; wb, wsc, p, \xi_0, \delta) \\ &= \mathrm{FC}_{wsc} \circ \hat{B}_p \circ \mathrm{Conv1}_{\xi_0,\delta}(x) + \mathrm{FC}_{wb} \circ \mathrm{SiLU}(x), \end{aligned} \tag{26}$$

here, FC denotes a fully connected layer:

$$[\mathrm{FC}_w(x)]_i = \sum_j w_{ij} x_j, \tag{27}$$

and Conv1 denotes a one-dimensional convolution with kernel size 1:

$$[\mathrm{Conv1}_{\xi_0,\delta}(x)]_{vj} = \frac{x_j}{\delta} - \left(v + \frac{\xi_0}{\delta}\right). \tag{28}$$

Theorem 3. *For a KAN Layer with uniform knots, let the order of the B-spline be p, and the knot points be:*

$$\xi_v = \xi_0 + v\delta, \tag{29}$$

then there exists a DATCNNKAN layer that is equivalent to it:

$$\begin{aligned} &\mathrm{KANLayer}(x; \xi, p, wb, ws, c) \\ &= \mathrm{DATCNNKANLayer}(x; wb, wsc, p, \xi_0, \delta), \end{aligned} \tag{30}$$

here:

$$wsc = \mathrm{reshape}(ws \otimes c, [d_O, nd_I]). \tag{31}$$

Proof. According to Definitions 2 and 5, and Theorem 2:

$$\begin{aligned} &[\mathrm{KANLayer}(x; \xi, p, wb, ws, c)]_i \\ &= \sum_{j=0}^{d_I-1} \phi_{ij}(x_j; \xi, p, wb_{ij}, ws_{ij}, c_{ij}) \\ &= \sum_{j=0}^{d_I-1} \left[wb_{ij}\mathrm{SiLU}(x_j) + ws_{ij} \sum_{v=0}^{n-1} c_{ijv} B_{\xi,p,v}(x_j) \right] \\ &= \sum_{j=0}^{d_I-1} wb_{ij}\mathrm{SiLU}(x_j) + \sum_{j=0}^{d_I-1} \sum_{v=0}^{n-1} ws_{ij} c_{ijv} \hat{B}_p\left(\frac{x_j - \xi_0}{\delta} - v\right) \\ &= \left[\mathrm{FC}_{wb} \circ \mathrm{SiLU}(x) + \mathrm{FC}_{wsc} \circ \hat{B}_p \circ \mathrm{Conv1}_{\xi_0,\delta}(x)\right]_i \\ &= \left[\mathrm{DATCNNKANLayer}(x; wb, wsc, p, \xi_0, \delta)\right]_i. \end{aligned} \tag{32}$$

□

3.5. From DATCNNKAN to DATCNN

Although the DATCNNKAN architecture exhibits a certain degree of complexity, the convolutional, activation, and fully connected layers in its backbone are indispensable, as they directly ensure both symbolic regression capability and interpretability. To enhance the model while preserving its symbolic regression capabilities, we have refined DATCNNKAN in several key aspects. First, we replace the normalized B-function with the SiLU activation function, which offers superior derivative properties. Second, inspired by ResNet (He et al., 2016), we streamline the residual connections to facilitate gradient propagation: an identity mapping is used when the input and output dimensions match, whereas a single fully connected layer is used when they differ. Third, to stabilize training, we apply batch normalization after the convolutional layers in the main branch and after the fully connected layers in the residual path. Finally, the convolutional layers in the main branch can be stacked. These enhancements culminate in the DATCNN architecture described in Section 3.1.

Compared with KAN, DATCNN introduces several key architectural modifications. First, it eliminates KAN's grid structure, replacing its functionality with convolutional layers to enable gradient-based optimization. Second, the parameterized recursive form of the original B-splines is replaced with a fixed SiLU activation, significantly reducing computational costs. Furthermore, the dual-path architecture of DATCNN includes batch normalization, the SiLU activation function, and residual connections to improve gradient stability. With these structural changes, DATCNN preserves the core advantages of KAN, specifically its symbolic regression capability and high interpretability.

4. Experiment

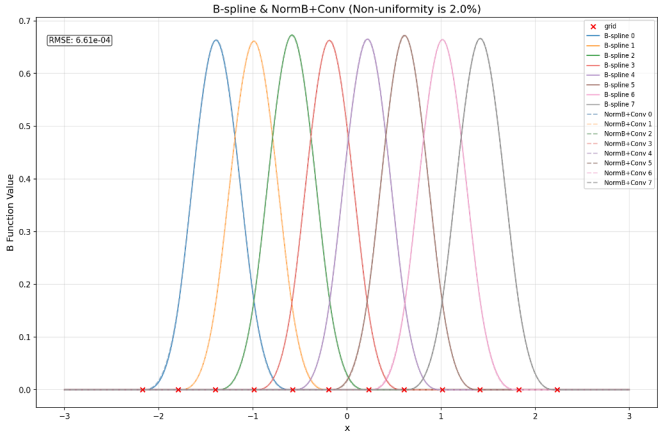
The experiment consists of three parts. In Section 4.1, we verify Theorems 2 and 3 by numerical simulation to check the correctness

Table 1
Relative errors of B-spline.

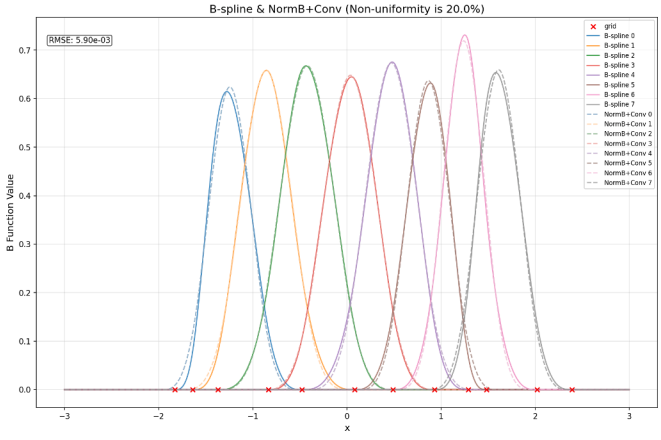
δ	ξ_0	p	n	Relative Error ↓
0.4	−2	1	5	6.3e-31
0.1	−1.5	3	20	1.5e-29
0.02	−1.2	5	100	2.5e-26
0.005	−1.2	10	500	7.2e-18

Table 2
Relative errors between KAN layer and DATCNNKAN layer.

d_I	d_O	p	n	Relative Error ↓
1	1	1	5	0
3	2	3	20	1.0e-31
10	5	5	100	3.6e-29
100	20	10	500	5.3e-18



(a) Non-uniformity is 2%.



(b) Non-uniformity is 20%.

Fig. 5. Illustration of approximation error in the case of a non-uniform grid. Fig. 5a shows the case with 2% non-uniformity, and Fig. 5b shows the case with 20% non-uniformity.

of the theoretical derivation. In Section 4.2, following the experimental design in the KAN paper, we compare the function fitting capabilities of KAN and DATCNN. In Section 4.3.1 and 4.3.2, we evaluate the effectiveness of DATCNN in common image classification and natural language processing tasks, demonstrating its flexibility and application value.

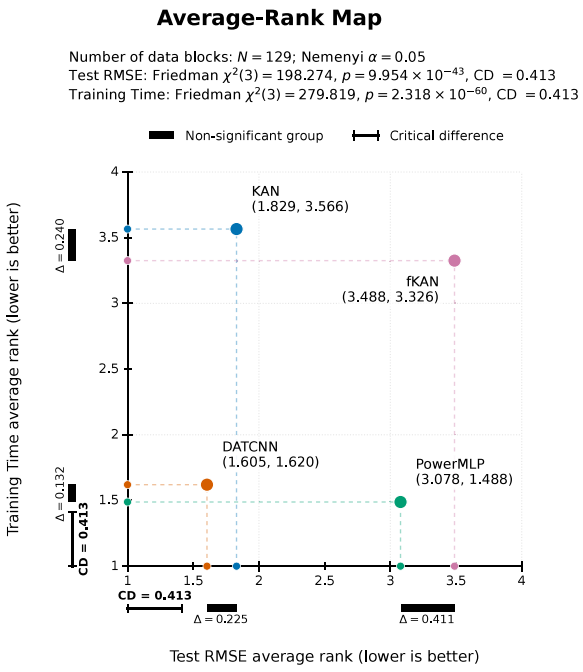


Fig. 6. Two-metric average-rank map for test RMSE and training time on the combined Feynman and special-function datasets. Lower average ranks are better. Capped rulers show the Nemenyi critical difference (CD) at $\alpha = 0.05$, and thick bars connect models without significant pairwise differences.

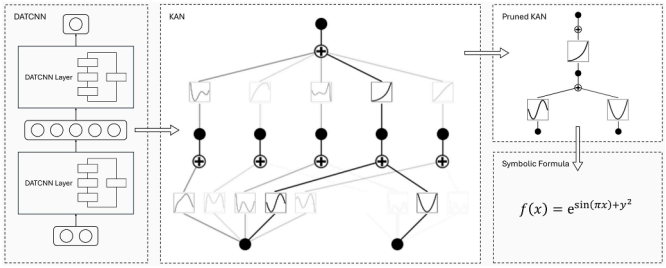


Fig. 7. A demonstration of symbolic regression: first, train a DATCNN on a dataset; then extract the backbone parameters of the DATCNN, interpret them as learnable activation functions, and convert the model into a KAN; finally, follow KAN's symbolic regression workflow.

4.1. Simulation validation of mathematical theory

We first verify Theorem 2. For B-splines with different uniform grids and different orders, we compute both sides of Eq. (19) using Definition 1 and Proposition 2, respectively. We use a random array of length 100,000 as input and then compare the relative errors. The results on both sides of the equation are arrays of shape $100,000 \times n$, and the relative error is defined as the ratio of the mean squared error between the two arrays to the sum of squares of a single array.

Table 1 presents the relative error between the left and right sides of formula (19). In the table header, δ, ξ_0, and n are the parameters of the uniform grid, as detailed in Eq. (12), and p is the order of the B-spline, as defined in Definition 1. Although the relative error gradually increases with the increase of p and n, it remains within the rounding error range, confirming the correctness of Theorem 2.

Next, we verify Theorem 3. For different numbers of grids, orders of B-spline, and layer shapes in the layer, we first instantiate a KAN Layer, migrate the parameters from the KAN Layer to the DATCNNKAN

Table 3
Function fitting results on Feynman and special function datasets.

Dataset	Model	Avg. RMSE ↓	Std. RMSE ↓	Avg. Time(s) ↓	Std. Time(s) ↓
Feynman	KAN	0.0032	0.0138	639	473
	fKAN	0.0225	0.1954	403	153
	PowerMLP	0.0084	0.0368	**128**	**48**
	DATCNN	**0.0025**	**0.0043**	152	102
Special Function	KAN	**0.0027**	0.0124	448	360
	fKAN	0.0180	0.2297	474	148
	PowerMLP	0.0467	0.1200	**124**	**34**
	DATCNN	0.0034	**0.0096**	174	135

Table 4
Ablation study on Feynman I.8.4.

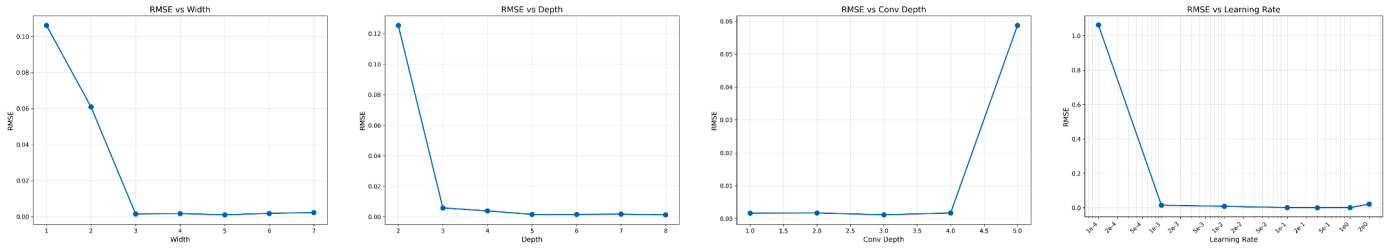
model	Depth of Conv. Layer	Activation Function	BatchNorm	RMSE ↓	Training Time (s) ↓
KAN	–	–	–	0.0056	570
DATCNN	1	NormB	False	0.0050	202
	1	NormB	True	0.0039	186
	1	SiLU	True	0.0025	**169**
	2	SiLU	True	**0.0023**	240

Table 5
Results of the hyperparameter sensitivity experiments on Feynman I.6.20b. Each group varies one hyperparameter while keeping the others at the baseline configuration. Lower RMSE is better. Light-gray cells indicate relatively stable regimes for multi-valued hyperparameters or the lower-RMSE setting in each binary comparison.

Network Width		Network Depth		Conv. Depth		Learning Rate		Residual Connection		BatchNorm	
Value	RMSE ↓	Value	RMSE ↓	Value	RMSE ↓	Value	RMSE ↓	Value	RMSE ↓	Value	RMSE ↓
1	0.1062	2	0.1255	1	0.0017	10^{-4}	1.0624	True	0.0012	True	0.0013
2	0.0610	3	0.0059	2	0.0017	10^{-3}	0.0154	False	0.0014	False	0.0027
3	0.0015	4	0.0039	3	0.0012	10^{-2}	0.0091	–	–	–	–
4	0.0017	5	0.0016	4	0.0017	10^{-1}	0.0016	–	–	–	–
5	0.0010	6	0.0016	5	0.0588	0.3	0.0011	–	–	–	–
6	0.0018	7	0.0018	–	–	1	0.0016	–	–	–	–
7	0.0022	8	0.0014	–	–	2	0.0216	–	–	–	–

Table 6
Image classification results on fashion-MNIST and SVHN datasets.

Dataset	Size	Kernel	Accuracy ↑	Precision ↑	Recall ↑	F1 Score ↑	Params ↓	Training Time (s/epoch) ↓
Fashion MNIST	Small	KAN	0.8816	0.8814	0.8816	0.8814	15,300	68
		PowerMLP	0.8791	0.8782	0.8791	0.8781	19,290	**40**
		fKAN	0.8860	0.8853	0.8860	0.8850	11,280	85
		DATCNN	**0.8876**	**0.8868**	**0.8876**	**0.8870**	14,790	47
	Medium	KAN	0.8880	0.8867	0.8880	0.8866	28,250	107
		PowerMLP	0.8917	0.8919	0.8917	0.8916	35,565	**55**
		fKAN	0.8937	0.8944	0.8937	0.8938	20,880	130
		DATCNN	**0.8946**	**0.8950**	**0.8946**	**0.8942**	27,315	69
	Big	KAN	0.9027	0.9029	0.9027	0.9026	51,850	107
		PowerMLP	0.8986	0.8983	0.8986	0.8981	59,165	**55**
		fKAN	0.8996	0.9005	0.8996	0.8999	44,480	130
		DATCNN	**0.9067**	**0.9061**	**0.9067**	**0.9060**	50,915	69
SVHN	Small	KAN	0.7924	0.7765	0.7680	0.7708	20,530	106
		PowerMLP	0.8442	0.8315	0.8284	0.8294	25,850	**53**
		fKAN	0.8533	0.8391	**0.8385**	0.8385	15,170	93
		DATCNN	**0.8551**	**0.8478**	0.8359	**0.8411**	19,850	60
	Medium	KAN	0.8317	0.8181	0.8113	0.8139	34,030	151
		PowerMLP	0.8526	0.8376	0.8378	0.8369	42,675	**73**
		fKAN	0.8642	0.8531	0.8493	0.8507	25,320	131
		DATCNN	**0.8716**	**0.8625**	**0.8585**	**0.8600**	32,925	82
	Big	KAN	0.8634	0.8518	0.8499	0.8504	67,530	152
		PowerMLP	0.8756	0.8647	0.8643	0.8642	76,175	**75**
		fKAN	0.8841	**0.8767**	0.8704	**0.8729**	58,820	138
		DATCNN	**0.8844**	0.8715	**0.8759**	0.8727	66,425	82



(a) Width. (b) Depth. (c) Convolutional layer depth. (d) Learning rate.

Fig. 8. Experiments on hyperparameter sensitivity, including network width, network depth, convolutional layer depth, and learning rate.

Table 7
Ablation study on FashionMNIST. The model size is small.

kernel	Size of Conv. Kernel	Depth of Conv. Layer	Activation Function	F1 Score ↑	Params ↓	Training Time (s/epoch) ↓
KAN	–	–	–	0.8814	15,300	68
DATCNN	1	1	NormB	0.8749	4890	53
	1	1	SiLU	0.8731	4890	**41**
	1	2	SiLU	0.8774	8190	45
	3	2	SiLU	**0.8870**	14,790	47

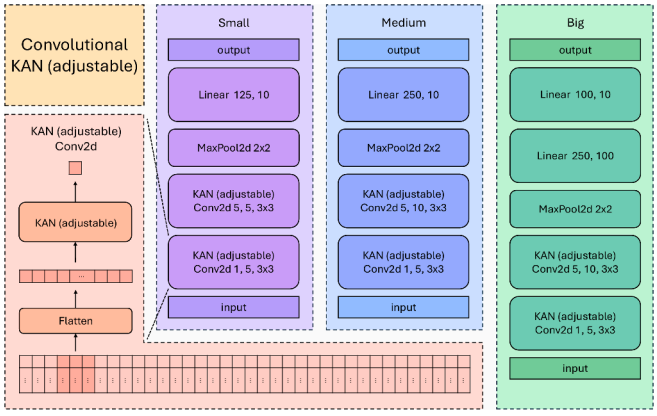


Fig. 9. The structural diagram of convolutional KAN. Convolutional KAN is available in three sizes, all of which feature a similar structure. The core part is the KAN Conv2d layer, located at the bottom left of the figure. Like a regular convolutional layer, the KAN Conv2d layer has a rectangular window that slides over the input. However, instead of performing a dot product with the convolutional kernel, the elements within the window are flattened and passed through a KAN layer to obtain the output.

Layer, and then compute both sides of Eq. (30) using Definitions 2 and 5, respectively. We use a random array of shape $1000 \times d_I$ (d_I is the input dimension of the layer) as input and compare their relative errors. The results on both sides of Eq. (30) are arrays of shape $1000 \times d_O$ (d_O is the output dimension of the layer), and the relative error is defined as the ratio of the mean squared error between the two arrays to the sum of squares of a single array.

Table 2 presents the relative error between the left and right sides of formula (30). In the table header, d_I and d_O represent the input and output dimensions of the KAN Layer or DATCNNKAN Layer, as detailed in Definitions 2 and 5, p is the order of the B-spline, as defined in Definition 1, and n is the parameter of the uniform grid, as detailed in Eq. (12). Although the relative error gradually increases with the increase of d_I, d_O, p, n, it always remains within the range of rounding errors, confirming the correctness of Theorem 3.

We evaluated the approximation error between B-splines and normalized B-functions on non-uniform grids. In the standard KAN implementation, the default grid non-uniformity is set to 2%. At this level, the approximation error (measured by RMSE) is approximately 6.61e-4. As illustrated in Fig. 5a, the discrepancy between the two curves is visually negligible. Even when grid non-uniformity is increased to 20%, the approximation error remains minimal at 5.90e-3 (see Fig. 5b), with the curves retaining close alignment. These results empirically demonstrate that the mathematical conclusions derived in Section 3 can be verified, when grid non-uniformity does not exceed 20%.

4.2. Function fitting

This part of the experiment compares the performance of KAN and DATCNN in function-fitting tasks. Following the experimental design outlined in the KAN paper (Liu et al., 2025), we utilize the special functions datasets (Liu et al., 2025) and the Feynman datasets (Udrescu & Tegmark, 2020). The special function dataset compiles common special functions in mathematics and physics, whereas the Feynman dataset gathers physics equations from Feynman's textbooks. The input and output dimensions of the model are adapted to the data, while all other intermediate dimensions are set to 5. We optimize for 200 steps in each stage using an LBFGS optimizer with a learning rate of 1. We search for the optimal configuration within the model depth range of $\{2, 3, 4, 5, 6\}$. To enable a fair comparison with fKAN (Aghaei, 2025) and PowerMLP (Qiu et al., 2025), the DATCNN and KAN are trained without grid extension.

Table 3 summarizes the function fitting results on the Feynman and special functions datasets. Fig. 6 complements the table with a two-metric average-rank map. For each retained function, the four models are ranked separately by test RMSE and training time, and the horizontal and vertical coordinates are the corresponding ranks averaged over the 129 function blocks; lower ranks indicate better performance. For each metric, the Friedman test assesses whether the models differ overall, while the Nemenyi critical difference (CD) identifies pairwise differences: a rank gap larger than the CD is statistically significant at $\alpha = 0.05$, whereas models joined by a thick bar do not differ significantly.

Taken together, Table 3 and Fig. 6 show that DATCNN and KAN do not differ significantly in test RMSE, whereas DATCNN significantly outperforms PowerMLP and fKAN. In training time, DATCNN and

Table 8
Text classification on TREC-6 and TREC-50 datasets.

Dataset	Model	Accuracy ↑	Precision ↑	Recall ↑	F1 Score ↑	Training Time (s) ↓
TREC-6	KAN	0.9260	0.9366	0.9364	0.9353	5985
	fKAN	**0.9600**	**0.9686**	0.9496	0.9583	101
	PowerMLP	0.9520	0.9601	0.9599	0.9594	**24**
	DATCNN	0.9560	0.9626	**0.9628**	**0.9625**	31
TREC-50	KAN	0.7200	0.6057	0.5335	0.5175	16,839
	fKAN	0.5180	0.3735	0.4138	0.3365	123
	PowerMLP	0.7120	0.6302	0.6098	0.5839	**25**
	DATCNN	**0.7500**	**0.6645**	**0.6302**	**0.6186**	32

Table 9
Semantic similarity regression on STS-B dataset.

Model	MAE ↓	RMSE ↓	Pearson ↑	Spearman ↑	Training Time (s) ↓
KAN	1.0800	1.2992	0.5527	0.5610	19,562
fKAN	0.5697	0.7155	0.8801	0.8950	191
PowerMLP	0.3762	0.5046	0.9420	0.9432	**40**
DATCNN	**0.2725**	**0.3478**	**0.9759**	**0.9759**	67

PowerMLP do not differ significantly, whereas DATCNN is significantly faster than KAN and fKAN. Table 3 further shows that DATCNN has the lowest RMSE standard deviation, indicating superior stability. Consequently, DATCNN offers the most favorable trade-off between accuracy and efficiency.

Detailed function-fitting results are provided in Appendix A. Tables A.10 and A.11 report the results for the Feynman and special-function datasets, respectively. Table 4 presents an ablation study of DATCNN on Formula I.8.4 from the Feynman dataset, validating the effectiveness of improvements in convolutional layer depth, activation functions, and batch normalization.

We analyzed the hyperparameter sensitivity of DATCNN. Starting from a baseline configuration (network width of 5, depth of 6, convolutional depth of 2, and a learning rate of 1, incorporating residual connections and batch normalization), we adjusted each hyperparameter individually. Using the Feynman dataset (formula I.6.20b) as a case study, we monitored fluctuations in the fitting error. The complete numerical results are summarized in Table 5, while the four multi-valued hyperparameters are illustrated in Fig. 8. DATCNN exhibits stability across a broad range of hyperparameter settings, thereby validating the rationale behind our experimental design.

Compared with improved variants such as fKAN and PowerMLP, DATCNN retains KAN's symbolic regression capability and interpretability. Fig. 7 gives an example of symbolic regression: to uncover the functional form underlying a dataset, we first fit the data using DATCNN, then extract its backbone parameters, interpret them as learnable activation functions, convert them into a KAN, and finally follow KAN's symbolic regression pipeline. This approach combines the advantages of DATCNN's fast training with the strong interpretability of symbolic systems.

4.3. Common task

This section validates DATCNN's performance on common tasks in image classification and natural language processing, demonstrating its flexibility and practical value.

4.3.1. Image classification

This section evaluates the effectiveness of DATCNN in common image classification tasks. Convolutional KAN (Bodner et al., 2024) is the first work to apply KAN to image classification, with a structure as shown in Fig. 9. Convolutional KAN is available in three sizes, all of which feature a similar structure. The core part is the KAN Conv2d Layer, at the bottom left of Fig. 9. Similar to a regular convolutional layer, the KAN Conv2d Layer has a rectangular window that slides over the input. However, instead of performing a dot product with the convolutional kernel, the elements within the window are flattened and passed through a KAN Layer to obtain the output. In this experiment, we retained the framework structure of Convolutional KAN and replaced the KAN layer with DATCNN layers, fKAN, and PowerMLP. To follow the experiments of Convolutional KAN (Bodner et al., 2024) and PowerMLP (Qiu et al., 2025), we selected the Fashion-MNIST (Xiao et al., 2017) and the Street View House Numbers (SVHN) (Netzer et al., 2011) datasets. Fashion-MNIST is a dataset containing images of Zalando's products, which are associated with labels from 10 categories. SVHN contains over 600,000 labeled digits cropped from Street View images. All hyperparameters were set to the original settings of Convolutional KAN, that is, a hyperparameter search was conducted within the range of learning rates $\{1\times10^{-3}, 5\times10^{-4}, 1\times10^{-4}\}$, weight decay $\{0, 1\times10^{-5}, 1\times10^{-4}\}$, batch sizes $\{128, 64, 32\}$, and a maximum of 20 training epochs.

Table 6 summarizes the comparative results, demonstrating that DATCNN possesses significant advantages in both classification performance and training efficiency. Furthermore, Table 7 presents an ablation study on the small-sized network. These results validate the efficacy of refinements in convolutional kernel size, layer depth, and activation functions, underscoring DATCNN's flexibility and broad application potential.

4.3.2. Natural language processing

This section evaluates the performance of DATCNN on natural language processing tasks using the TREC-6, TREC-50 (Li & Roth, 2002), and STS-B (Cer et al., 2017; Wang et al., 2018a) benchmarks. Derived from the TREC QA Track (Voorhees & Dang, 2001), TREC-6 and TREC-50 classify questions into six coarse and 50 fine-grained semantic categories, respectively. These datasets comprise 5450 training, 500 validation, and 500 test samples. The STS-B dataset assesses semantic textual similarity between sentence pairs (5,750 training, 1500 validation, 1380 test samples) by requiring the model to predict a similarity score ranging from 0 (no overlap) to 5 (equivalence). Performance is evaluated based on the correlation between model predictions and human judgments. Regarding input representation, we employ averaged GloVe embeddings (Pennington et al., 2014) for classification tasks and sentence-transformer embeddings (Reimers & Gurevych, 2019) for regression tasks. Consistent with the function-fitting experiments, we set the model width to 5 and the depth search space to 2–6. All models are trained for 10 epochs using the Adam optimizer with a learning rate of 1×10^{-3}.

Tables 8 and 9 summarize the experimental results for the natural language processing tasks. Notably, DATCNN achieves superior performance while maintaining high training efficiency. In contrast, KAN's training time is orders of magnitude longer than that of other models. This discrepancy is attributed to the 300-dimensional embedding used in this experiment, which is significantly higher than the input

dimensions in prior studies. These findings reveal inherent limitations in KAN's computational graph, leading to a drastic surge in computational costs as the input dimension increases.

5. Conclusion

In this paper, we presented DATCNN, a novel convolutional neural network architecture that effectively integrates the core advantages of KAN. By refining a convolutional structure that is theoretically equivalent to KAN, DATCNN preserves KAN's interpretability and symbolic regression capabilities while significantly enhancing architectural flexibility. We provided a theoretical justification for the proposed architecture via mathematical analysis and validated its effectiveness through extensive simulations. Empirical evaluations across diverse scenarios, including function fitting, image classification, and natural language processing, demonstrate that DATCNN outperforms KAN and its variants in both predictive performance and training efficiency. These results highlight the broad applicability of DATCNN as a flexible, interpretable deep learning backbone.

CRediT authorship contribution statement

Ruikun Luo: Writing – original draft, Visualization, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Nan Su:** Writing – review & editing, Writing – original draft, Validation, Supervision, Methodology, Conceptualization; **Yixiang Dai:** Writing – review & editing, Validation; **Guosheng Li:** Validation; **Guijin Wang:** Writing – review & editing, Supervision, Resources, Project administration, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Result of function fitting experiments

Table A.10
Result of Feynman dataset.

Num.	Function	Model	RMSE	Training Time(s)
I.6.20a	$\frac{1}{\sqrt{2\pi}}e^{-\frac{\theta^2}{2}}$	KAN	0.0004	672.8
		fKAN	0.0006	611.0
		PowerMLP	0.0065	85.4
		DATCNN	**0.0004**	**24.5**
I.6.20	$\frac{1}{\sqrt{2\pi\sigma^2}}e^{-\frac{\theta^2}{2\sigma^2}}$	KAN	**0.0004**	227.0
		fKAN	0.0011	479.9
		PowerMLP	0.0149	103.8
		DATCNN	0.0012	**69.8**
I.6.20b	$\frac{1}{\sqrt{2\pi\sigma^2}}e^{-\frac{(\theta-\theta_1)^2}{2\sigma^2}}$	KAN	0.0027	258.2
		fKAN	0.1412	348.6
		PowerMLP	0.0260	**115.6**
		DATCNN	**0.0012**	211.5
I.8.4	$\sqrt{(x_2-x_1)^2+(y_2-y_1)^2}$	KAN	0.0056	570.9
		fKAN	0.4974	412.0
		PowerMLP	0.0464	**146.8**
		DATCNN	**0.0023**	240.3
I.9.18	$\frac{Gm_1m_2}{(x_2-x_1)^2+(y_2-y_1)^2+(z_2-z_1)^2}$	KAN	**0.0014**	826.0
		fKAN	0.0278	254.0
		PowerMLP	0.0022	**39.0**
		DATCNN	0.0018	106.7
I.10.7	$\frac{m_0}{\sqrt{1-\frac{v^2}{c^2}}}$	KAN	0.0090	285.0
		fKAN	0.3535	491.2
		PowerMLP	0.0040	89.3
		DATCNN	**0.0018**	**78.0**
I.11.19	$x_1y_1+x_2y_2+x_3y_3$	KAN	0.0034	1934.8
		fKAN	0.0071	637.3
		PowerMLP	0.0038	**153.3**
		DATCNN	**0.0028**	208.1
I.12.1	μN_n	KAN	**0.0005**	758.5
		fKAN	0.0013	429.8
		PowerMLP	0.0015	**65.9**
		DATCNN	0.0009	109.5
I.12.2	$\frac{q_1q_2}{4\pi\epsilon r^2}$	KAN	0.0047	734.2
		fKAN	**0.0017**	365.1
		PowerMLP	0.0033	**102.2**
		DATCNN	0.0022	150.0
I.12.4	$\frac{q_1}{4\pi\epsilon r^2}$	KAN	**0.0009**	173.4
		fKAN	0.0625	242.9
		PowerMLP	0.0063	108.0
		DATCNN	0.0013	**51.9**
I.12.5	q_2E_f	KAN	**0.0005**	741.4
		fKAN	0.0013	356.3
		PowerMLP	0.0015	**64.7**
		DATCNN	0.0009	107.2
I.12.11	$q(E_f+Bv\sin(\theta))$	KAN	0.0974	1788.8
		fKAN	0.0104	295.2
		PowerMLP	0.1541	**203.5**
		DATCNN	**0.0097**	392.9
I.13.4	$\frac{1}{2}m(v^2+u^2+w^2)$	KAN	0.0014	348.9
		fKAN	0.0047	538.4
		PowerMLP	0.0017	147.8
		DATCNN	**0.0011**	**80.2**
I.13.12	$Gm_1m_2\left(\frac{1}{r_2}-\frac{1}{r_1}\right)$	KAN	0.0022	285.0
		fKAN	0.0051	263.0
		PowerMLP	0.0033	**157.0**
		DATCNN	**0.0019**	174.7
I.14.3	mgz	KAN	**0.0007**	234.2
		fKAN	0.0095	472.9
		PowerMLP	0.0018	**64.0**
		DATCNN	0.0014	118.9
I.14.4	$\frac{1}{2}k_sx^2$	KAN	**0.0004**	604.5
		fKAN	0.0008	66.1
		PowerMLP	0.0008	66.7
		DATCNN	0.0009	**50.3**

Table A.10
Table A.10 (continued).

I.15.3x	$\frac{x-ut}{\sqrt{1-\frac{u^2}{c^2}}}$	KAN	0.0304	748.6
		fKAN	0.0657	506.8
		PowerMLP	0.0210	**107.6**
		DATCNN	**0.0083**	326.1
I.15.3t	$\frac{t-\frac{ux}{c^2}}{\sqrt{1-\frac{u^2}{c^2}}}$	KAN	0.0356	960.7
		fKAN	0.0494	469.0
		PowerMLP	0.0315	**204.1**
		DATCNN	**0.0141**	215.2
I.15.10	$\frac{m_0 v}{\sqrt{1-\frac{v^2}{c^2}}}$	KAN	0.0029	337.5
		fKAN	0.0051	118.8
		PowerMLP	**0.0026**	**112.9**
		DATCNN	0.0030	125.3
I.16.6	$\frac{u+v}{1+\frac{uv}{c^2}}$	KAN	**0.0013**	295.1
		fKAN	0.0039	140.5
		PowerMLP	0.0042	127.3
		DATCNN	0.0028	**90.9**
I.18.4	$\frac{m_1 r_1+m_2 r_2}{m_1+m_2}$	KAN	0.0010	798.5
		fKAN	0.0015	461.3
		PowerMLP	0.0019	99.0
		DATCNN	**0.0009**	**86.4**
I.18.4	$rF\sin(\theta)$	KAN	0.0040	1225.0
		fKAN	0.2358	**232.4**
		PowerMLP	0.1531	276.3
		DATCNN	**0.0021**	243.9
I.18.16	$mrv\sin(\theta)$	KAN	0.0744	1837.3
		fKAN	**0.0048**	494.6
		PowerMLP	0.0973	**213.9**
		DATCNN	0.0058	370.4
I.24.6	$\frac{1}{4}m(\omega^2+\omega_0^2)x^2$	KAN	0.0013	322.8
		fKAN	0.0020	688.9
		PowerMLP	**0.0012**	136.5
		DATCNN	0.0022	**97.5**
I.25.13	$\frac{q}{C}$	KAN	0.0011	244.0
		fKAN	0.0036	384.1
		PowerMLP	0.0027	**96.7**
		DATCNN	**0.0011**	125.7
I.26.2	$\arcsin(n\sin(\theta_2))$	KAN	**0.0034**	410.3
		fKAN	0.0068	611.6
		PowerMLP	0.0566	**145.5**
		DATCNN	0.0037	297.6
I.27.6	$\frac{1}{\frac{1}{d_1}+\frac{n}{d_2}}$	KAN	**0.0006**	743.5
		fKAN	0.1794	624.2
		PowerMLP	0.0057	88.0
		DATCNN	0.0011	**44.4**
I.29.4	$\frac{\omega}{c}$	KAN	**0.0007**	729.6
		fKAN	0.0042	500.8
		PowerMLP	0.0084	**78.2**
		DATCNN	0.0009	114.0
I.29.16	$\sqrt{x_1^2+x_2^2-2x_1x_2\cos(\theta_1-\theta_2)}$	KAN	0.2056	1704.1
		fKAN	0.0519	519.6
		PowerMLP	0.2466	**212.3**
		DATCNN	**0.0328**	444.1
I.30.3	$\frac{I_0\sin^2\left(\frac{n\theta}{2}\right)}{\sin^2\left(\frac{\theta}{2}\right)}$	KAN	**0.0131**	1602.8
		fKAN	0.3961	496.9
		PowerMLP	0.2964	**144.2**
		DATCNN	0.0167	395.5
I.30.5	$\arcsin\left(\frac{\lambda}{nd}\right)$	KAN	**0.0009**	696.3
		fKAN	0.0046	469.0
		PowerMLP	0.0045	144.6
		DATCNN	0.0011	**69.2**
I.32.5	$\frac{q^2a^2}{\epsilon c^3}$	KAN	0.0404	1034.0
		fKAN	0.3057	546.4
		PowerMLP	0.0580	209.8
		DATCNN	**0.0062**	**182.4**

Table A.10
Table A.10 (continued).

I.32.17	$\frac{1}{2}\epsilon c E_f^2\left(\frac{8\pi r^2}{3}\right)\frac{\omega^4}{(\omega^2-\omega_0^2)^2}$	KAN	0.2768	1408.1
		fKAN	0.2626	424.9
		PowerMLP	0.1451	**190.9**
		DATCNN	**0.0741**	373.4
I.34.8	$\frac{qVB}{p}$	KAN	0.0083	1283.7
		fKAN	0.0126	316.4
		PowerMLP	0.0044	**130.6**
		DATCNN	**0.0040**	255.6
I.34.10	$\frac{\omega_0}{1-\frac{v}{c}}$	KAN	0.0022	440.5
		fKAN	0.5585	595.6
		PowerMLP	0.0021	**116.2**
		DATCNN	**0.0019**	135.8
I.34.14	$\omega_0\frac{1+\frac{v}{c}}{\sqrt{1-\frac{v^2}{c^2}}}$	KAN	**0.0013**	280.0
		fKAN	0.4018	683.7
		PowerMLP	0.0033	**117.3**
		DATCNN	0.0014	126.5
I.34.27	$\hbar\omega$	KAN	**0.0005**	755.1
		fKAN	0.0013	284.1
		PowerMLP	0.0015	**61.9**
		DATCNN	0.0009	93.8
I.37.4	$I_1+I_2+2\sqrt{I_1I_2}\cos(\delta)$	KAN	**0.0024**	646.7
		fKAN	0.8559	456.6
		PowerMLP	0.1935	**161.4**
		DATCNN	0.0030	233.6
I.38.12	$\frac{4\pi\epsilon\hbar^2}{mq^2}$	KAN	0.1084	1614.9
		fKAN	0.8757	272.1
		PowerMLP	0.0981	**135.0**
		DATCNN	**0.0369**	329.8
I.39.10	$\frac{3}{2}p_F V$	KAN	**0.0005**	589.9
		fKAN	0.0035	197.8
		PowerMLP	0.0015	112.5
		DATCNN	0.0008	**45.9**
I.39.11	$\frac{p_F V}{\gamma-1}$	KAN	0.0013	253.2
		fKAN	0.2627	490.8
		PowerMLP	0.0041	121.0
		DATCNN	**0.0011**	**74.5**
I.39.22	$\frac{nk_bT}{V}$	KAN	0.0012	817.0
		fKAN	0.0030	391.1
		PowerMLP	0.0028	**111.8**
		DATCNN	**0.0012**	163.3
I.40.1	$n_0 e^{-\frac{mgx}{k_bT}}$	KAN	0.0043	818.1
		fKAN	0.2891	575.4
		PowerMLP	0.0064	**132.9**
		DATCNN	**0.0033**	281.4
I.41.16	$\frac{\hbar\omega^3}{\pi^2c^2\left(e^{\frac{\hbar\omega}{k_bT}}-1\right)}$	KAN	**0.0011**	256.9
		fKAN	0.0024	208.5
		PowerMLP	0.0015	168.6
		DATCNN	0.0018	**59.1**
I.43.16	$\frac{\mu q V_e}{d}$	KAN	0.0012	795.8
		fKAN	0.0030	350.6
		PowerMLP	0.0028	**101.1**
		DATCNN	**0.0012**	137.8
I.43.31	$\mu k_b T$	KAN	**0.0007**	199.9
		fKAN	0.0265	647.8
		PowerMLP	0.0022	**68.1**
		DATCNN	0.0011	109.0
I.43.43	$\frac{k_b v}{A(\gamma-1)}$	KAN	0.0072	347.2
		fKAN	0.0062	333.4
		PowerMLP	0.0225	153.2
		DATCNN	**0.0028**	**91.3**
I.44.4	$nk_bT\log\left(\frac{V_2}{V_1}\right)$	KAN	**0.0017**	310.7
		fKAN	0.0029	633.1
		PowerMLP	0.0036	**160.3**
		DATCNN	0.0019	180.1
I.47.23	$\sqrt{\frac{\gamma p}{\rho}}$	KAN	**0.0009**	746.2
		fKAN	0.2126	421.2
		PowerMLP	0.0027	171.4
		DATCNN	0.0011	**136.1**

Table A.10
Table A.10 (continued).

I.48.20	$\frac{mc^2}{\sqrt{1-\frac{v^2}{c^2}}}$	KAN	**0.0019**	416.6
		fKAN	0.8881	626.7
		PowerMLP	0.0030	232.2
		DATCNN	0.0030	**173.3**
I.50.26	$x_1(\cos(\omega t)+\alpha\cos^2(\omega t))$	KAN	0.0456	1375.4
		fKAN	0.0174	637.3
		PowerMLP	0.1239	**156.9**
		DATCNN	**0.0067**	352.0
II.2.42	$\frac{\kappa(T_2-T_1)A}{d}$	KAN	0.0018	317.9
		fKAN	0.0034	487.6
		PowerMLP	0.0033	**77.5**
		DATCNN	**0.0015**	121.0
II.3.24	$\frac{P}{4\pi r^2}$	KAN	**0.0002**	126.0
		fKAN	0.0050	261.6
		PowerMLP	0.0061	65.9
		DATCNN	0.0009	**48.2**
II.4.23	$\frac{q}{4\pi\epsilon r}$	KAN	**0.0005**	639.9
		fKAN	0.0010	312.2
		PowerMLP	0.0051	86.1
		DATCNN	0.0008	**37.9**
II.6.11	$\frac{1}{4\pi\epsilon}\frac{p_d\cos(\theta)}{r^2}$	KAN	0.0035	260.7
		fKAN	0.0041	372.8
		PowerMLP	0.0232	219.5
		DATCNN	**0.0021**	**94.5**
II.6.15a	$\frac{3}{4\pi\epsilon}\frac{p_d z}{r^5}\sqrt{x^2+y^2}$	KAN	0.0418	1279.1
		fKAN	0.0680	303.4
		PowerMLP	0.0439	**143.7**
		DATCNN	**0.0100**	260.2
II.6.15b	$\frac{3}{4\pi\epsilon}\frac{p_d}{r^3}\cos(\theta)\sin(\theta)$	KAN	0.0171	650.3
		fKAN	0.0083	333.6
		PowerMLP	0.0975	**101.2**
		DATCNN	**0.0064**	183.4
II.8.7	$\frac{3}{5}\frac{q^2}{4\pi\epsilon d}$	KAN	**0.0005**	138.9
		fKAN	0.0015	253.9
		PowerMLP	0.0037	83.1
		DATCNN	0.0007	**34.0**
II.8.31	$\frac{1}{2}\epsilon E_f^2$	KAN	**0.0002**	586.5
		fKAN	0.0010	165.3
		PowerMLP	0.0009	83.5
		DATCNN	0.0007	**37.8**
I.10.9	$\frac{\sigma}{\epsilon(1+\chi)}$	KAN	**0.0007**	217.4
		fKAN	0.2146	338.2
		PowerMLP	0.0086	140.8
		DATCNN	0.0011	**43.2**
II.11.3	$\frac{qE_f}{m(\omega_0^2-\omega^2)}$	KAN	0.0076	247.5
		fKAN	0.0075	414.5
		PowerMLP	0.0125	102.2
		DATCNN	**0.0021**	**70.0**
II.11.17	$n_0\left(1+\frac{p_d E_f\cos(\theta)}{k_b T}\right)$	KAN	0.0459	1816.0
		fKAN	0.0416	627.1
		PowerMLP	0.1177	**142.5**
		DATCNN	**0.0092**	358.1
II.11.20	$\frac{np_d^2 E_f}{3k_b T}$	KAN	**0.0010**	718.2
		fKAN	0.0027	272.5
		PowerMLP	0.0015	100.0
		DATCNN	0.0018	**61.6**
II.11.27	$\frac{n\alpha}{1-\frac{n\alpha}{3}}\epsilon E_f$	KAN	0.0191	829.4
		fKAN	0.0131	719.1
		PowerMLP	0.0069	**114.6**
		DATCNN	**0.0044**	167.2
II.11.28	$1+\frac{n\alpha}{1-\frac{n\alpha}{3}}$	KAN	**0.0009**	250.8
		fKAN	0.0085	427.3
		PowerMLP	0.0033	**83.3**
		DATCNN	0.0014	121.5
II.13.17	$\frac{1}{4\pi\epsilon c^2}\frac{2I}{r}$	KAN	0.0117	223.2
		fKAN	0.0051	**95.2**
		PowerMLP	0.0312	207.0
		DATCNN	**0.0028**	115.4

Table A.10
Table A.10 (continued).

II.13.23	$\frac{\rho}{\sqrt{1-\frac{v^2}{c^2}}}$	KAN	0.0090	257.0
		fKAN	0.3535	457.3
		PowerMLP	0.0040	86.1
		DATCNN	**0.0018**	**73.8**
II.13.34	$\frac{\rho v}{\sqrt{1-\frac{v^2}{c^2}}}$	KAN	0.0051	320.5
		fKAN	0.3109	478.0
		PowerMLP	**0.0030**	**86.0**
		DATCNN	0.0049	193.2
II.15.4	$-\mu_M B\cos(\theta)$	KAN	0.0019	305.2
		fKAN	0.0192	391.7
		PowerMLP	0.0619	214.9
		DATCNN	**0.0017**	**100.4**
II.15.5	$-p_d E_f\cos(\theta)$	KAN	0.0019	308.9
		fKAN	0.0192	390.2
		PowerMLP	0.0619	203.2
		DATCNN	**0.0017**	**98.5**
II.21.32	$\frac{q}{4\pi\epsilon r\left(1-\frac{v}{c}\right)}$	KAN	0.0174	243.6
		fKAN	0.0215	216.5
		PowerMLP	**0.0112**	**98.9**
		DATCNN	0.0113	155.4
II.24.17	$\sqrt{\frac{\omega^2}{c^2}-\frac{\pi^2}{d^2}}$	KAN	0.0010	638.9
		fKAN	0.2612	305.0
		PowerMLP	0.0094	163.3
		DATCNN	**0.0009**	**120.6**
II.27.16	$\epsilon c E_f^2$	KAN	**0.0013**	615.9
		fKAN	0.1269	149.5
		PowerMLP	0.0019	**63.7**
		DATCNN	0.0019	79.7
II.27.18	ϵE_f^2	KAN	**0.0004**	160.2
		fKAN	0.0022	311.5
		PowerMLP	0.0015	64.6
		DATCNN	0.0010	**53.0**
II.34.2a	$\frac{qv}{2\pi r}$	KAN	**0.0006**	157.5
		fKAN	0.0395	275.9
		PowerMLP	0.0009	67.4
		DATCNN	0.0010	**36.3**
II.34.2	$\frac{qvr}{2}$	KAN	**0.0005**	175.1
		fKAN	0.0724	151.0
		PowerMLP	0.0012	67.1
		DATCNN	0.0014	**41.0**
II.34.11	$\frac{gqB}{2m}$	KAN	0.0009	207.1
		fKAN	0.0022	182.3
		PowerMLP	0.0027	85.3
		DATCNN	**0.0008**	**52.3**
II.34.29a	$\frac{qh}{4\pi m}$	KAN	**0.0003**	116.3
		fKAN	0.0007	206.7
		PowerMLP	0.0004	62.0
		DATCNN	0.0006	**36.8**
II.34.29b	$\frac{g\mu BJ}{h}$	KAN	0.0019	212.0
		fKAN	0.0027	310.3
		PowerMLP	0.0039	**69.3**
		DATCNN	**0.0017**	85.5
II.35.18	$\frac{n_0}{e^{\frac{\mu B}{k_b T}}+e^{-\frac{\mu B}{k_b T}}}$	KAN	**0.0010**	209.5
		fKAN	0.0034	316.4
		PowerMLP	0.0023	164.0
		DATCNN	0.0015	**46.7**
II.35.21	$n\mu\tanh\left(\frac{\mu B}{k_b T}\right)$	KAN	0.0030	821.9
		fKAN	0.0055	517.4
		PowerMLP	0.0041	**117.8**
		DATCNN	**0.0026**	125.0
II.36.38	$\frac{\mu B}{k_b T}+\frac{\mu\alpha M}{\epsilon c^2 k_b T}$	KAN	0.1381	1855.3
		fKAN	0.1078	501.6
		PowerMLP	0.0359	**129.8**
		DATCNN	**0.0085**	181.8
II.37.1	$\mu(1+\chi)B$	KAN	**0.0010**	221.8
		fKAN	0.3009	364.4
		PowerMLP	0.0036	**71.6**
		DATCNN	0.0014	89.1

Table A.10
Table A.10 (continued).

II.38.3	$\frac{YAx}{d}$	KAN	0.0012	792.7
		fKAN	0.0030	387.2
		PowerMLP	0.0028	**101.8**
		DATCNN	**0.0012**	132.4
II.38.14	$\frac{Y}{2(1+\sigma)}$	KAN	**0.0003**	607.0
		fKAN	0.0010	177.5
		PowerMLP	0.0004	108.4
		DATCNN	0.0008	**37.1**
III.4.32	$\frac{1}{e^{\frac{h\omega}{k_bT}}-1}$	KAN	**0.0023**	529.5
		fKAN	1.6992	508.2
		PowerMLP	0.0062	142.1
		DATCNN	0.0026	**133.6**
III.4.33	$\frac{h\omega}{e^{\frac{h\omega}{k_bT}}-1}$	KAN	0.0018	330.9
		fKAN	0.0043	443.6
		PowerMLP	0.0037	98.0
		DATCNN	**0.0018**	**97.7**
III.7.38	$\frac{2\mu B}{h}$	KAN	0.0022	393.4
		fKAN	0.4968	207.8
		PowerMLP	0.0020	**110.8**
		DATCNN	**0.0014**	118.1
III.8.54	$\sin^2\left(\frac{Et}{h}\right)$	KAN	**0.0183**	896.2
		fKAN	0.0622	424.7
		PowerMLP	0.2441	**211.7**
		DATCNN	0.0526	297.7
III.9.52	$\frac{p_d E_f t}{h}\frac{\sin^2\left(\frac{(\omega-\omega_0)t}{2}\right)}{\left(\frac{(\omega-\omega_0)t}{2}\right)^2}$	KAN	**0.0025**	477.2
		fKAN	0.0047	568.9
		PowerMLP	0.0150	**180.4**
		DATCNN	0.0028	231.0
III.10.19	$\mu\sqrt{B_x^2+B_y^2+B_z^2}$	KAN	0.0024	345.8
		fKAN	0.2883	597.6
		PowerMLP	0.0032	149.5
		DATCNN	**0.0022**	**118.0**
III.12.43	nh	KAN	**0.0003**	201.1
		fKAN	0.0019	488.3
		PowerMLP	0.0011	110.5
		DATCNN	0.0008	**42.9**
III.13.18	$\frac{2Ed^2k}{h}$	KAN	0.0022	332.2
		fKAN	0.0040	363.6
		PowerMLP	0.0030	**83.6**
		DATCNN	**0.0018**	138.3
III.14.14	$I_0\left(e^{\frac{qV_e}{k_bT}}-1\right)$	KAN	0.0377	340.8
		fKAN	0.0272	439.7
		PowerMLP	0.0124	206.9
		DATCNN	**0.0066**	**167.2**
III.15.12	$2U(1-\cos(kd))$	KAN	**0.0023**	1155.7
		fKAN	0.8773	426.5
		PowerMLP	0.2367	**139.7**
		DATCNN	0.0031	252.6
III.15.14	$\frac{h^2}{2Ed^2}$	KAN	0.0029	194.3
		fKAN	0.0056	335.2
		PowerMLP	0.0091	96.5
		DATCNN	**0.0013**	**79.8**
III.15.27	$\frac{2\pi\alpha}{nd}$	KAN	0.0118	650.8
		fKAN	2.3648	426.4
		PowerMLP	0.0744	**137.9**
		DATCNN	**0.0021**	140.6
III.17.37	$\beta(1+\alpha\cos(\theta))$	KAN	0.0024	802.0
		fKAN	0.1433	464.5
		PowerMLP	0.0395	179.8
		DATCNN	**0.0018**	**159.2**
III.19.51	$-\frac{mq^4}{2(4\pi\epsilon)^2h^2}\frac{1}{n^2}$	KAN	**0.0008**	119.0
		fKAN	0.0013	247.3
		PowerMLP	0.0011	87.9
		DATCNN	0.0012	**32.5**
III.21.20	$-\frac{\rho qA}{m}$	KAN	**0.0013**	673.7
		fKAN	0.0084	209.8
		PowerMLP	0.0032	102.5
		DATCNN	0.0013	**75.4**

Table A.10
Table A.10 (continued).

Rutherford scattering	$\left(\frac{Z_1Z_2\alpha\hbar c}{4E\sin^2\left(\frac{\theta}{2}\right)}\right)^2$	KAN	0.1286	1348.5
		fKAN	0.1303	510.9
		PowerMLP	0.0896	140.7
		DATCNN	**0.0126**	**129.0**
Friedmann equation	$\sqrt{\frac{8\pi G}{3}\rho-\frac{k_fc^2}{a_f^2}}$	KAN	0.0023	891.6
		fKAN	0.5975	604.4
		PowerMLP	0.0040	**98.0**
		DATCNN	**0.0015**	106.0
Compton scattering	$\frac{E}{1+\frac{E}{mc^2}(1-\cos(\theta))}$	KAN	0.0032	548.4
		fKAN	0.0033	701.1
		PowerMLP	0.0557	225.6
		DATCNN	**0.0024**	**141.3**
Relativistic aberration	$\arccos\left(\frac{\cos(\theta_2)-\frac{v}{c}}{1-\frac{v}{c}\cos(\theta_2)}\right)$	KAN	0.0073	375.9
		fKAN	0.9117	398.9
		PowerMLP	0.0106	157.8
		DATCNN	**0.0045**	**102.5**
N-slit diffraction	$I_0\left(\frac{\sin\left(\frac{\alpha}{2}\right)}{\frac{\alpha}{2}}\frac{\sin\left(\frac{N\delta}{2}\right)}{\sin\left(\frac{\delta}{2}\right)}\right)^2$	KAN	**0.0012**	263.9
		fKAN	0.0019	234.5
		PowerMLP	0.0020	126.7
		DATCNN	0.0015	**53.0**
Goldstein 3.16	$\sqrt{\frac{2}{m}\left(E-U-\frac{L^2}{2mr^2}\right)}$	KAN	**0.0012**	321.5
		fKAN	0.2337	594.2
		PowerMLP	0.0023	157.3
		DATCNN	0.0014	**56.5**
Goldstein 3.55	$\frac{mk_G}{L^2}\left(1+\sqrt{1+\frac{2EL^2}{mk_G^2}}\cos(\theta_1-\theta_2)\right)$	KAN	0.1341	2217.7
		fKAN	2.6119	338.6
		PowerMLP	1.6879	**142.6**
		DATCNN	**0.0492**	445.7
Goldstein 3.64 (ellipse)	$\frac{d(1-\alpha^2)}{1+\alpha\cos(\theta_2-\theta_1)}$	KAN	0.0166	1364.0
		fKAN	0.0218	578.2
		PowerMLP	0.1140	**139.1**
		DATCNN	**0.0048**	264.9
Goldstein 3.74 (Kepler)	$\frac{2\pi d^{\frac{3}{2}}}{\sqrt{G(m_1+m_2)}}$	KAN	**0.0024**	782.3
		fKAN	1.2260	241.5
		PowerMLP	0.0123	**89.2**
		DATCNN	0.0031	129.8
Goldstein 3.99	$\sqrt{1+\frac{2\epsilon^2EL^2}{m(Z_1Z_2q^2)^2}}$	KAN	0.0365	470.0
		fKAN	0.2136	**108.4**
		PowerMLP	0.0633	134.6
		DATCNN	**0.0057**	140.3
Goldstein 8.56	$\sqrt{(p-qA)^2c^2+m^2c^4}+qV_e$	KAN	0.0096	1266.1
		fKAN	0.3116	433.0
		PowerMLP	0.0192	**146.5**
		DATCNN	**0.0079**	358.5
Goldstein 12.80	$\frac{1}{2m}\left(p^2+m^2\omega^2x^2\left(1+\frac{\alpha y}{x}\right)\right)$	KAN	0.0099	476.3
		fKAN	0.2102	371.5
		PowerMLP	0.0069	**101.5**
		DATCNN	**0.0042**	126.6
Jackson 2.11	$\frac{q}{4\pi\epsilon y^2}\left(4\pi\epsilon V_e d-\frac{qdy^3}{(y^2-d^2)^2}\right)$	KAN	0.0390	386.1
		fKAN	0.0705	275.7
		PowerMLP	0.0573	**96.4**
		DATCNN	**0.0237**	192.3
Jackson 4.60	$E_f\cos(\theta)\left(\frac{\alpha-1}{\alpha+2}\frac{d^3}{r^2}-r\right)$	KAN	0.0181	1164.5
		fKAN	0.0280	531.0
		PowerMLP	0.0131	**201.4**
		DATCNN	**0.0110**	280.7
Jackson 11.38 (Doppler)	$\frac{\sqrt{1-\frac{v^2}{c^2}}}{1+\frac{v}{c}\cos(\theta)}\omega$	KAN	0.0292	883.9
		fKAN	0.0099	621.6
		PowerMLP	0.0114	**244.1**
		DATCNN	**0.0052**	359.2
Weinberg 15.2.1	$\frac{3}{8\pi G}\left(\frac{c^2k_f}{a_f^2}+H^2\right)$	KAN	0.0021	284.9
		fKAN	0.0037	507.6
		PowerMLP	0.0020	**170.5**
		DATCNN	**0.0020**	296.9
Weinberg 15.2.2	$-\frac{1}{8\pi G}\left(\frac{c^4k_f}{a_f^2}+c^2H^2(1-2\alpha)\right)$	KAN	0.0028	249.1
		fKAN	0.0028	373.6
		PowerMLP	0.0029	**90.0**
		DATCNN	**0.0022**	281.9

Table A.11
Result of special function dataset.

Name	Function	Model	RMSE	Training Time(s)
Jacobian elliptic functions	$(\mathrm{sn}, \mathrm{cn}, \mathrm{dn}, \phi)(u, m)$	KAN	**0.0005**	307.1
		fKAN	0.0660	819.6
		PowerMLP	0.0025	71.8
		DATCNN	0.0010	**52.8**
Incomplete elliptic integral of the first kind	$\int_0^\phi \frac{d\theta}{\sqrt{1-m\sin^2\theta}}$	KAN	0.0826	1252.3
		fKAN	1.8214	368.8
		PowerMLP	0.3356	**142.1**
		DATCNN	**0.0305**	474.2
Incomplete elliptic integral of the second kind	$\int_0^\phi \sqrt{1-m\sin^2\theta}d\theta$	KAN	**0.0016**	735.6
		fKAN	1.5523	428.8
		PowerMLP	0.0816	**85.6**
		DATCNN	0.0033	177.4
Bessel function of the first kind	$J_n(x)$	KAN	**0.0033**	288.3
		fKAN	0.0128	614.5
		PowerMLP	0.0364	181.7
		DATCNN	0.0049	**153.5**
Modified Bessel function of the first kind	$I_n(x)$	KAN	**0.0052**	197.7
		fKAN	0.0081	450.9
		PowerMLP	0.0216	166.6
		DATCNN	0.0055	**166.5**
Associated Legendre function ($m = 0$)	$P_l^0(x)$	KAN	0.0117	397.3
		fKAN	0.0482	488.5
		PowerMLP	0.0177	**123.6**
		DATCNN	**0.0073**	197.3
Associated Legendre function ($m = 1$)	$P_l^1(x)$	KAN	**0.0334**	1026.7
		fKAN	0.0885	621.2
		PowerMLP	0.1188	**125.6**
		DATCNN	0.0434	405.6
spherical harmonics ($m = 0, n = 1$)	$Y_1^0(\theta, \phi)$	KAN	**0.0004**	158.4
		fKAN	0.0010	326.3
		PowerMLP	0.0407	142.7
		DATCNN	0.0008	**58.4**
spherical harmonics ($m = 1, n = 1$)	$Y_1^1(\theta, \phi)$	KAN	**0.0009**	222.2
		fKAN	0.0014	326.7
		PowerMLP	0.0629	137.3
		DATCNN	0.0011	**89.3**
spherical harmonics ($m = 0, n = 2$)	$Y_2^0(\theta, \phi)$	KAN	**0.0005**	179.9
		fKAN	0.0026	327.3
		PowerMLP	0.0232	133.7
		DATCNN	0.0008	**51.6**
spherical harmonics ($m = 1, n = 2$)	$Y_2^1(\theta, \phi)$	KAN	**0.0011**	279.4
		fKAN	0.0022	441.9
		PowerMLP	0.1226	**86.3**
		DATCNN	0.0018	114.4
spherical harmonics ($m = 2, n = 2$)	$Y_2^2(\theta, \phi)$	KAN	**0.0018**	336.7
		fKAN	0.0019	474.3
		PowerMLP	0.1312	**92.8**
		DATCNN	0.0019	147.8

References

Aghaei, A. A. (2024). rKAN: Rational Kolmogorov-Arnold networks. arXiv:2406.14495.

Aghaei, A. A. (2025). fKAN: Fractional Kolmogorov–Arnold networks with trainable Jacobi basis functions. *Neurocomputing*, *623*, 129414.

Bien, J., & Tibshirani, R. (2011). Prototype selection for interpretable classification. *The Annals of Applied Statistics*, *5*(4), 2403–2424.

Bodner, A. D., Tepsich, A. S., Spolski, J. N., & Pourteau, S. (2024). Convolutional Kolmogorov-Arnold networks. arXiv:2406.13155.

Bresson, R., Nikolentzos, G., Panagopoulos, G., Chatzianastasis, M., Pang, J., & Vazirgiannis, M. (2024). KAGNNs: Kolmogorov-Arnold networks meet graph learning. arXiv:2406.18380.

Caruana, R., Kangarloo, H., Dionisio, J. D., Sinha, U., & Johnson, D. (1999). Case-based explanation of non-case-based learning methods. In *Proceedings of the AMIA symposium* (p. 212).

Cen, C., Li, F., Hu, P., & Li, Z. (2025). A multi-scale vision mixture-of-experts for salient object detection with Kolmogorov-Arnold adapter. *Neurocomputing*, *644*, 130349.

Cer, D., Diab, M., Agirre, E., Lopez-Gazpio, I., & Specia, L. (2017). SemEval-2017 task 1: Semantic textual similarity-multilingual and cross-lingual focused evaluation. arXiv:1708.00055.

Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, *2*(4), 303–314.

Erhan, D., Bengio, Y., Courville, A., & Vincent, P. (2009). Visualizing higher-layer features of a deep network (Technical Report No. 1341). Département d'informatique et de recherche opérationnelle, Université de Montréal.

Fakhoury, D., Fakhoury, E., & Speleers, H. (2022). ExSpliNet: An interpretable and expressive spline-based neural network. *Neural Networks*, *152*, 332–346.

Genet, R., & Inzirillo, H. (2024). TKAN: Temporal Kolmogorov-Arnold networks. arXiv:2405.07344.

Haykin, S. (1998). Neural networks: A comprehensive foundation. Prentice Hall PTR.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770–778).

Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, *2*(5), 359–366.

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, *25*, 1097–1105.

Li, C., Liu, X., Li, W., Wang, C., Liu, H., Liu, Y., Chen, Z., & Yuan, Y. (2025). U-KAN makes strong backbone for medical image segmentation and generation. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 4652–4660). (*vol. 39*).

Li, X., & Roth, D. (2002). Learning question classifiers. In *Coling 2002: The 19th international conference on computational linguistics*.

Liu, Z., Wang, Y., Vaidya, S., Ruehle, F., Halverson, J., Soljacic, M., Hou, T. Y., & Tegmark, M. (2025). KAN: Kolmogorov–Arnold networks. In *The thirteenth international conference on learning representations*.

Makke, N., & Chawla, S. (2024). Interpretable scientific discovery with symbolic regression: A review. *Artificial Intelligence Review*, *57*(1), 2.

Netzer, Y., Wang, T., Coates, A., Bissacco, A., Wu, B., Ng, A. Y. et al. (2011). Reading digits in natural images with unsupervised feature learning. In *Nips workshop on deep learning and unsupervised feature learning* (p. 7). Granada (*vol. 2011*).

Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)* (pp. 1532–1543).

Qiu, R., Miao, Y., Wang, S., Zhu, Y., Yu, L., & Gao, X.-S. (2025). PowerMLP: An efficient version of KAN. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 20069–20076). (*vol. 39*).

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using siamese bert-networks. arXiv:1908.10084.

Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE international conference on computer vision* (pp. 618–626).

Setiono, R., & Liu, H. (1995). Understanding neural networks via rule extraction. In *IJCAI* (pp. 480–485). CiteSeer (*vol. 1*).

Sidharth, S. S., Gokul, R., Anas, K. P., & Keerthana, A. R. (2024). Chebyshev polynomial-based Kolmogorov-Arnold networks: An efficient architecture for nonlinear function approximation. arXiv:2405.07200v3.

Simonyan, K., Vedaldi, A., & Zisserman, A. (2013). Deep inside convolutional networks: Visualising image classification models and saliency maps. arXiv:1312.6034.

Springenberg, J. T., Dosovitskiy, A., Brox, T., & Riedmiller, M. (2014). Striving for simplicity: The all convolutional net. arXiv:1412.6806.

Taghizadeh, M., Zandsalimi, Z., Nabian, M. A., Shafiee-Jood, M., & Alemazkoor, N. (2025). Interpretable physics-informed graph neural networks for flood forecasting. *Computer-Aided Civil and Infrastructure Engineering*, *40*(18), 2629–2649.

Udrescu, S.-M., & Tegmark, M. (2020). AI Feynman: A physics-inspired method for symbolic regression. *Science Advances*, *6*(16), eaay2631.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, *30*, 5998–6008.

Voorhees, E. M., & Dang, H. T. (2001). Overview of the TREC 2001 question answering track. In *TREC* (pp. 42–51).

Wang, A., Singh, A., Michael, J., Hill, F., Levy, O., & Bowman, S. (2018a). GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the 2018 EMNLP workshop blackboxNLP: Analyzing and interpreting neural networks for NLP* (pp. 353–355).

Wang, Y., Su, H., Zhang, B., & Hu, X. (2018b). Interpret neural networks by identifying critical data routing paths. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 8906–8914).

Xiao, H., Rasul, K., & Vollgraf, R. (2017). Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms. arXiv:1708.07747.

Zeiler, M. D., & Fergus, R. (2014). Visualizing and understanding convolutional networks. In *Computer vision–ECCV 2014: 13th european conference, Zurich, Switzerland, September 6–12, 2014, proceedings, Part I 13* (pp. 818–833). Springer.

Zhang, Y., Tiňo, P., Leonardis, A., & Tang, K. (2021). A survey on neural network interpretability. *IEEE Transactions on Emerging Topics in Computational Intelligence*, *5*(5), 726–742.